

경사 하강법 (Gradient Descent)

김성영교수
국립금오공과대학교
컴퓨터공학부

학습 내용

- 경사 하강법(Gradient Descent Method)에 대해 설명할 수 있다.
- 학습율(learning rate)의 역할을 설명할 수 있다.

Cost function 고찰 (b 생략)

$$J(w, b) = \frac{1}{n} \sum_{k=1}^n \left(\underset{\text{예측값}}{H(x^{(k)})} - \underset{\text{실제값}}{y^{(k)}} \right)^2$$

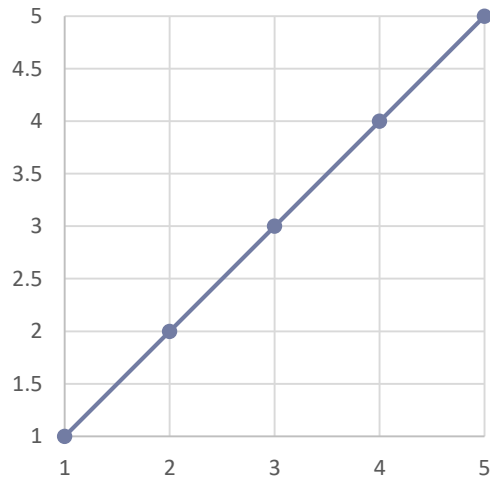
차이의 제곱을 평균낸 값 => 평균 제곱 오차 MSE

$$H(x) = \hat{y} = wx$$

$$J(w) = \frac{1}{n} \sum_{k=1}^n \left(wx^{(k)} - y^{(k)} \right)^2$$

w 선택

$x^{(k)}$	$y^{(k)}$
1	1
2	2
3	3
4	4
5	5



$$J(w) = \frac{1}{n} \sum_{k=1}^n (wx^{(k)} - y^{(k)})^2$$

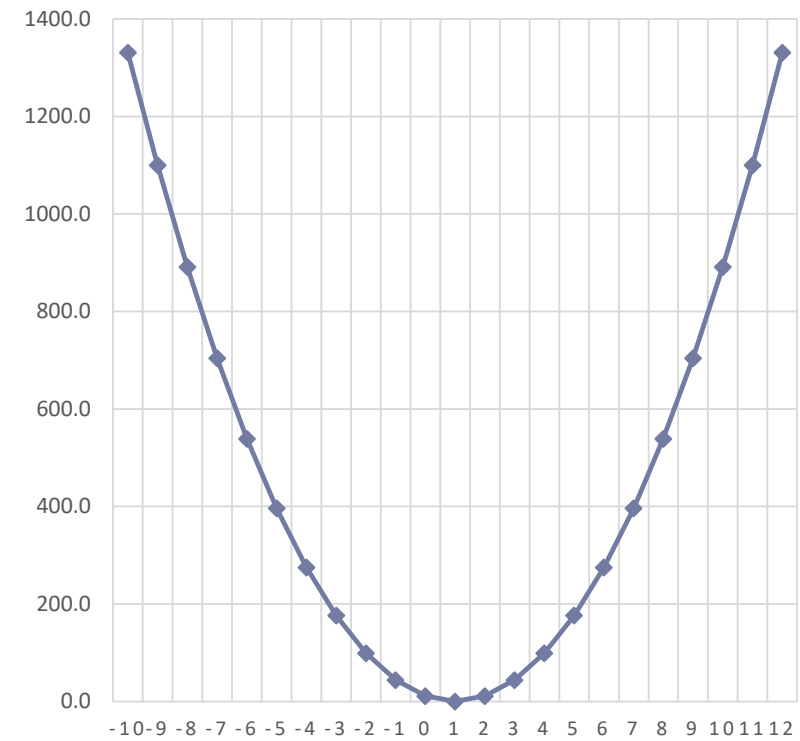
$$J(0) = 11$$

$$J(1) = 0$$

$$J(2) = 11$$

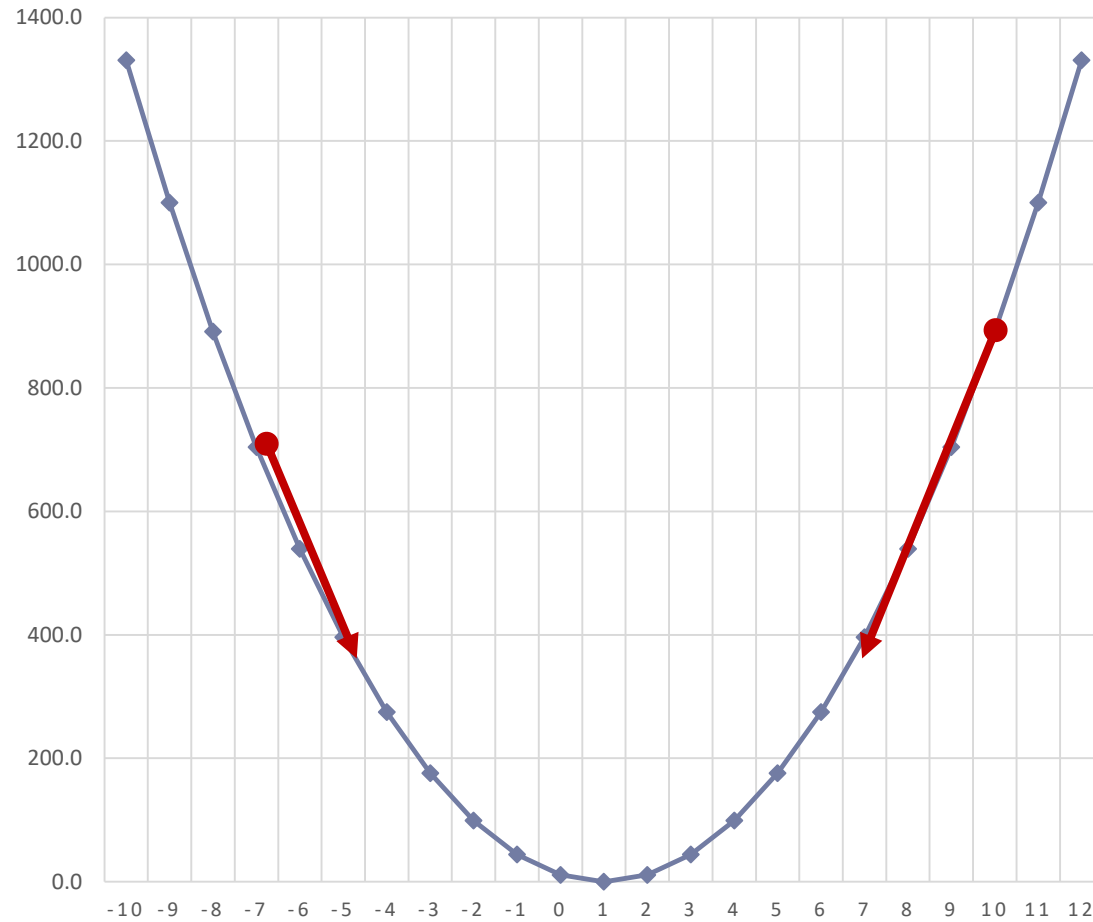
$$J(3) = 44$$

$$J(4) = 99$$



평균 제곱 오차에 대한 그래프

cost 최소화 방법



Gradient descent
algorithm

$$J(w, b) = \frac{1}{n} \sum_{k=1}^n (H(x^{(k)}) - y^{(k)})^2 \left(= (y^{(k)} - H(x^{(k)}))^2 \right)$$

잔차 벡터 (residual vector)

$$J(w, b) = \frac{1}{2n} \sum_{k=1}^n (H(x^{(k)}) - y^{(k)})^2$$

왜 2를 넣었을까?
나중에 계산할 때 숫자가 예쁘게 떨어지라고 일부러 2를 넣은 거야

$$w^{(t+1)} := w^{(t)} - \alpha \frac{\partial}{\partial w} J(w, b)$$

“지금 w 에서, 틀린 점수가 작아지는 방향으로 조금 이동해!”

α : Learning rate

$w^{(t)}$: 지금 가지고 있는 w

$w^{(t+1)}$: 조금 고친 다음 w

α (알파) : 얼마나 크게 움직일지 정하는 숫자 → 이게 학습률 (Learning rate) 이야. → 너무 크면 막 뛰어다녀서 오히려 더 틀릴 수 있고, → 너무 작으면 거북이처럼 느려.

$$w^{(t+1)} := w^{(t)} - \alpha \frac{\partial}{\partial w} \frac{1}{2n} \sum_{k=1}^n (H(x^{(k)}) - y^{(k)})^2$$

$H(x) = wx$

$$w^{(t+1)} := w^{(t)} - \alpha \frac{\partial}{\partial w} \frac{1}{2n} \sum_{k=1}^n (wx^{(k)} - y^{(k)})^2$$

원래는 이렇게 생겼어:
 “지금 w 에서, 틀린 점수가 줄어드는 방향으로 조금 이동해”
 그런데 여기에

$H(x) = w \times x$ 를 그냥 대입한 거야.

(예측값이 “ w 곱하기 x ”라는 뜻)

수학적으로 “미분”이라는 걸 해서 공식이 이렇게 바뀌었어:

“지금 예측한 값과 진짜 값의 차이”에 “ x ”를 곱한 것들을 평균 내서,
 그 방향으로 조금 이동해!
 쉽게 말하면:

각 데이터마다(내가 예측한 값 - 진짜 값) $\times x$ 를 계산해
 그 숫자들을 다 더해서 평균 내
 그 평균값에 α 를 곱해서
 지금 w 에서 빼버려

$$w^{(t+1)} := w^{(t)} - \alpha \frac{1}{n} \sum_{k=1}^n (wx^{(k)} - y^{(k)})x^{(k)}$$

(예측값 - 실제값)*입력값

이렇게 하면 틀린 점수가 줄어드는 쪽으로 w 가 조금씩 움직여.

경사 하강법 gradient descent (1)

- 현재 지점의 오차를 확인하여 오차가 작은 지점으로 반복 이동하여 오차가 최소가 되는 지점을 찾는 방법
- 오차가 작은 지점으로 이동하기 위해 미분 사용
- 알고리즘
 - step 1: 현재 위치에서 미분을 사용하여 기울기 계산
 - step 2: 기울기의 반대 방향으로 일정 값 만큼 이동
 - 기울기 +: 음의 방향(그래프 왼쪽 방향)으로 이동
 - 기울기 -: 양의 방향(그래프 오른쪽 방향)으로 이동
 - step 3: 미분 값이 0 또는 최소값이 아니면 step 1로 돌아감

경사 하강법 gradient descent (2)

- 초기 값으로 시작 (w, b)
- $cost(w, b)$ 가 감소하도록 w 와 b 를 조금씩 변경
- *local minimum*에 도달할 때까지 반복 수행
- 특징
 - 시작하는 위치에 따라 도달하는 최저점이 결정됨
(where you start can determine which minimum you end up)

$$\begin{aligned}
 & w^{(t+1)} := w^{(t)} - \alpha \frac{\partial}{\partial w} \frac{1}{2n} \sum_{k=1}^n \left(H(x^{(k)}) - y^{(k)} \right)^2 \left(= \left(y^{(k)} - H(x^{(k)}) \right)^2 \right) \\
 & \quad \text{잔차 벡터 (residual vector)} \\
 & H(x) = wx + b \\
 & w^{(t+1)} := w^{(t)} - \alpha \frac{\partial}{\partial w} \frac{1}{2n} \sum_{k=1}^n \left((wx^{(k)} + b) - y^{(k)} \right)^2
 \end{aligned}$$

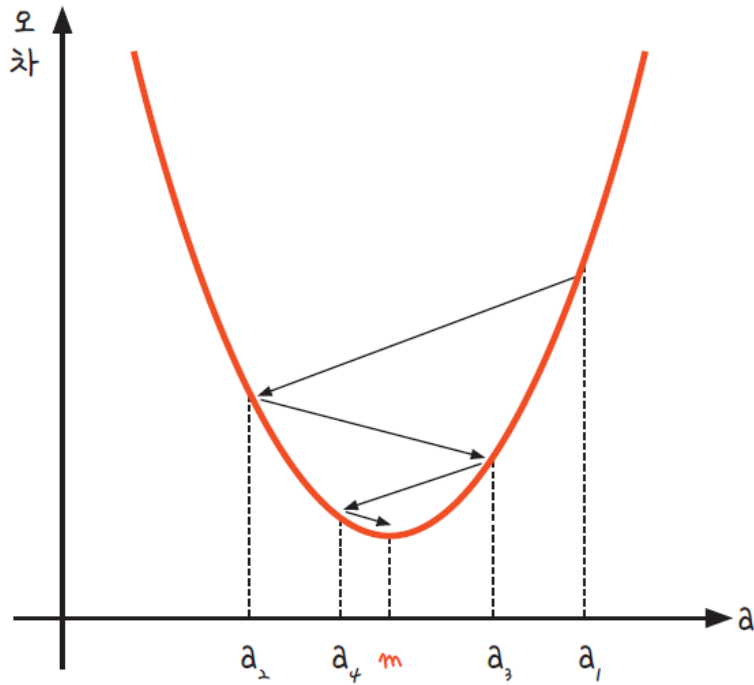
$$\left\{ \begin{aligned}
 & w^{(t+1)} := w^{(t)} - \alpha \frac{1}{n} \sum_{k=1}^n \left((wx^{(k)} + b) - y^{(k)} \right) x^{(k)} \\
 & b^{(t+1)} := b^{(t)} - \alpha \frac{1}{n} \sum_{k=1}^n \left((wx^{(k)} + b) - y^{(k)} \right)
 \end{aligned} \right.$$

batch gradient descent

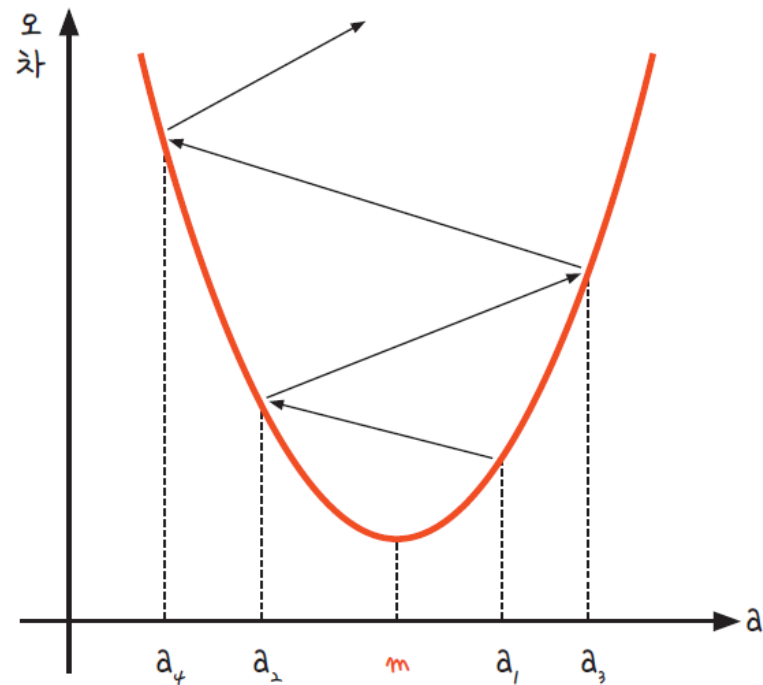
stochastic gradient descent
mini-batch gradient descent

학습률 learning rate

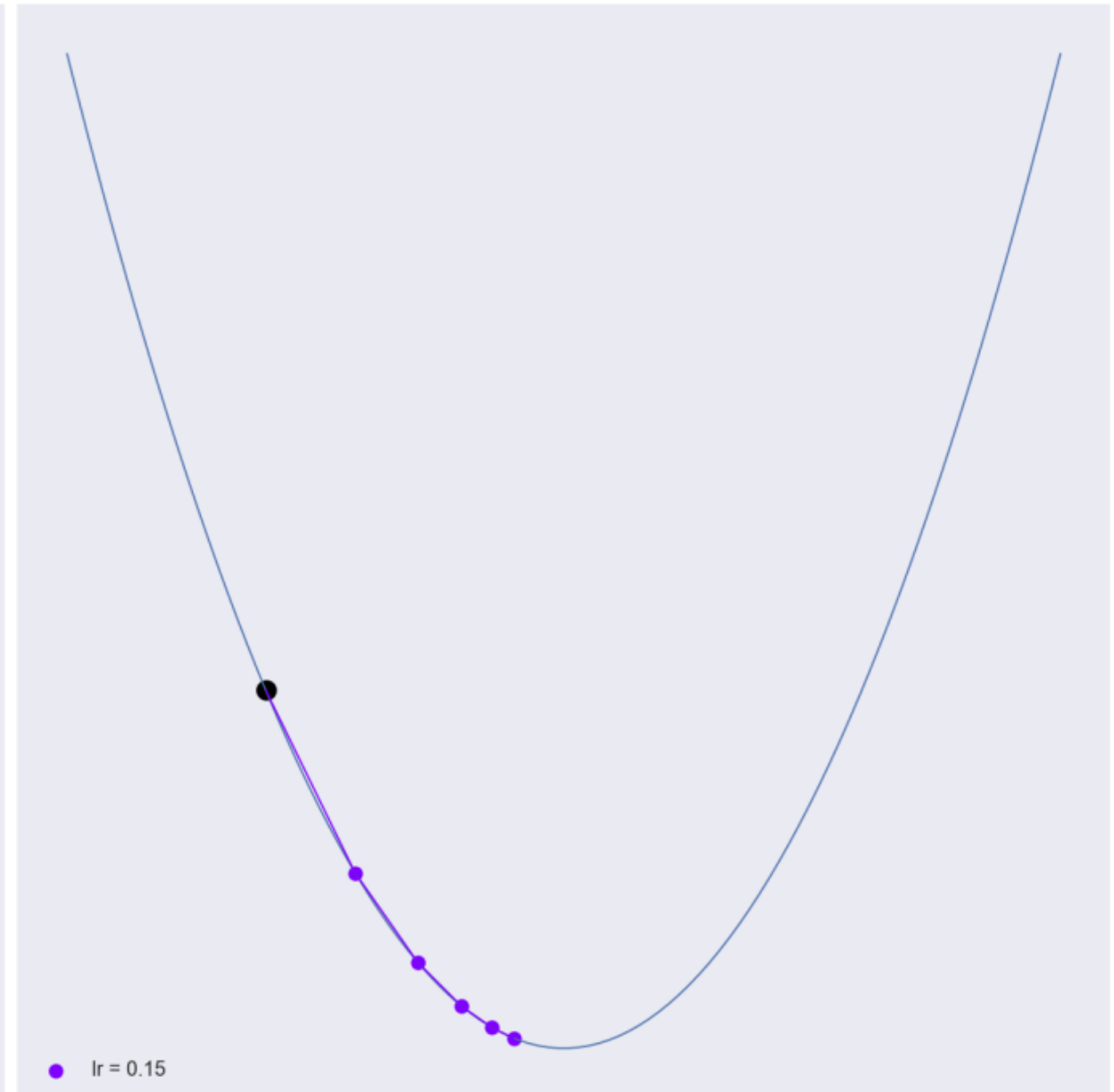
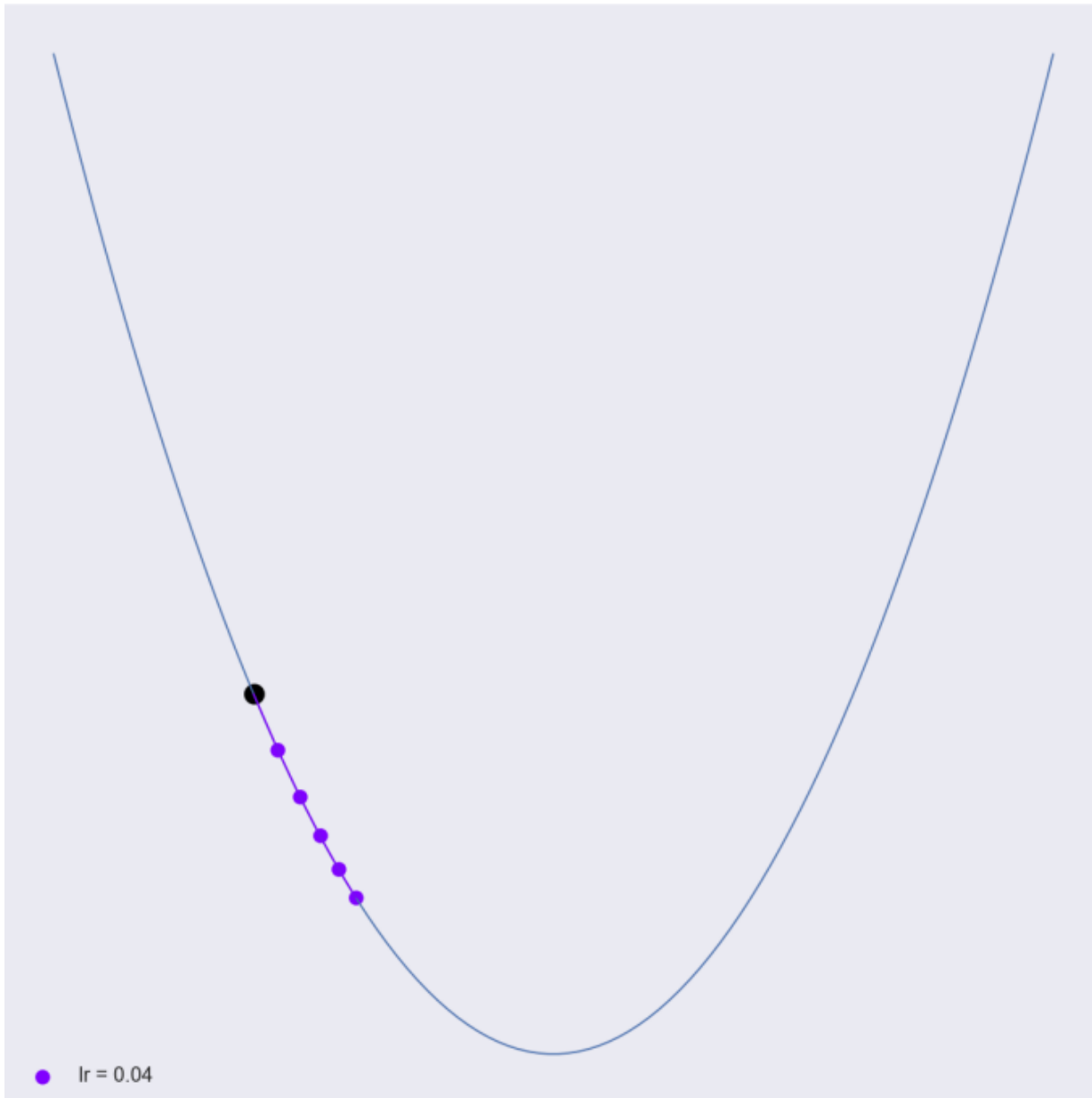
a 값

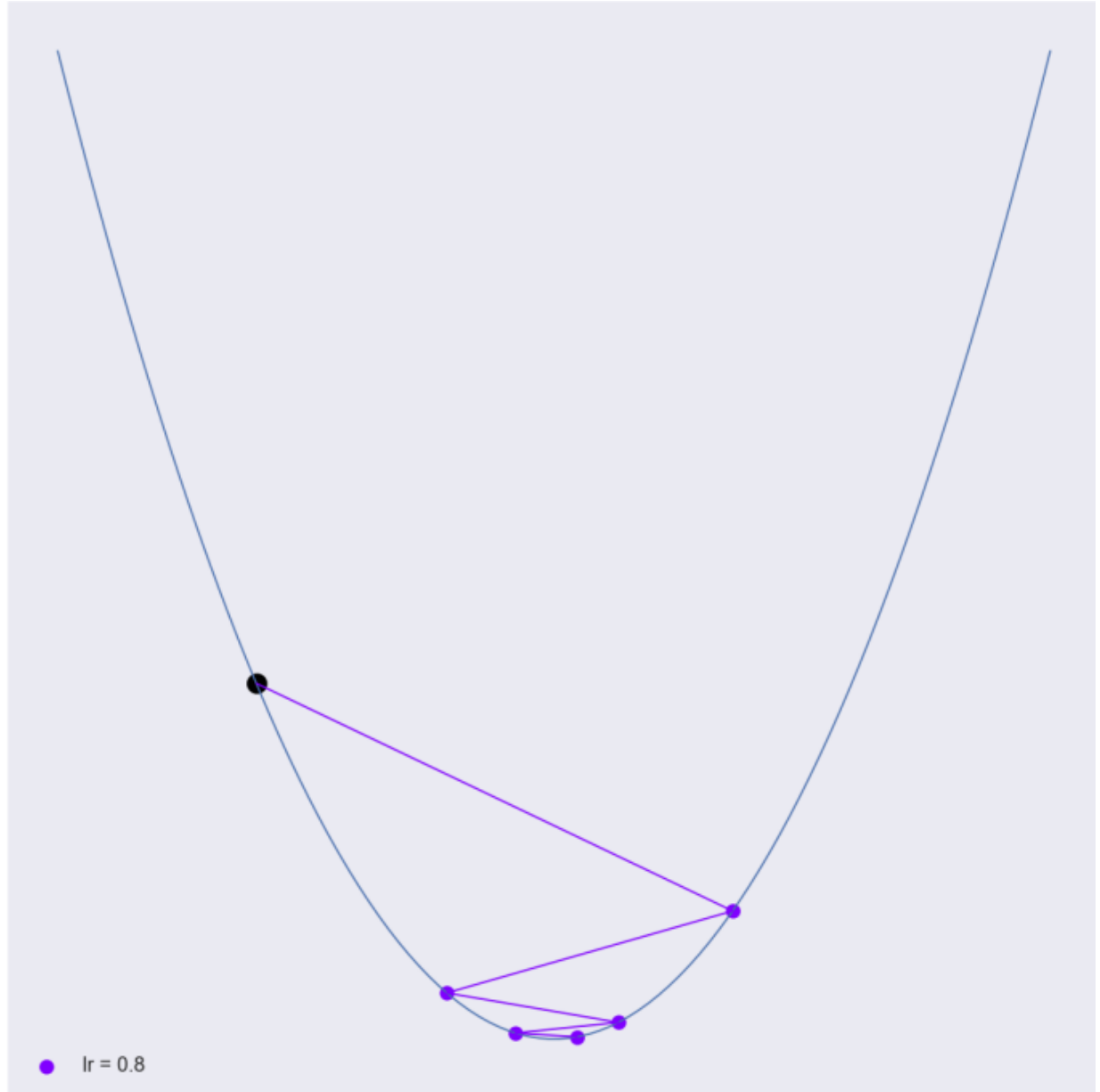
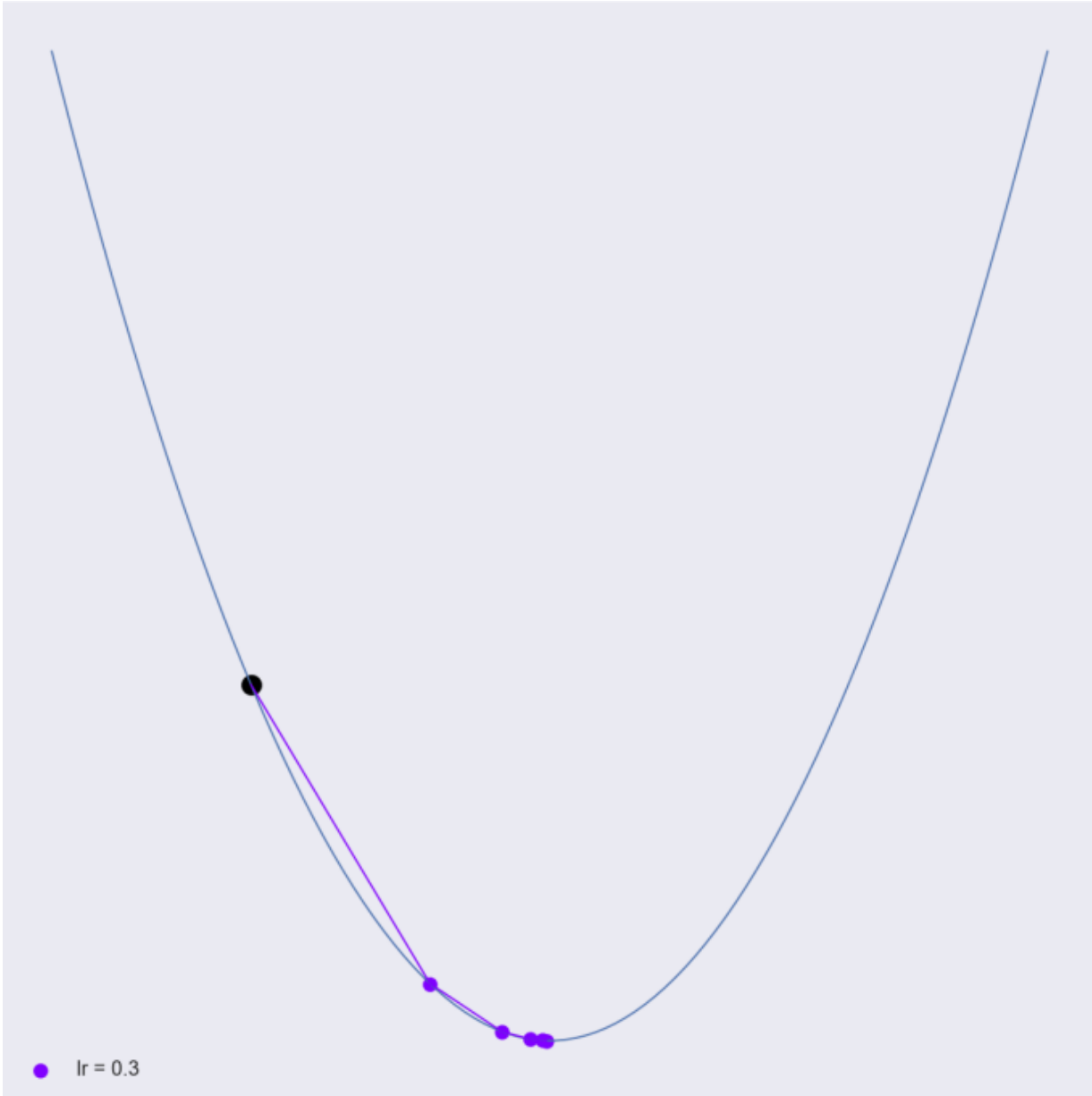


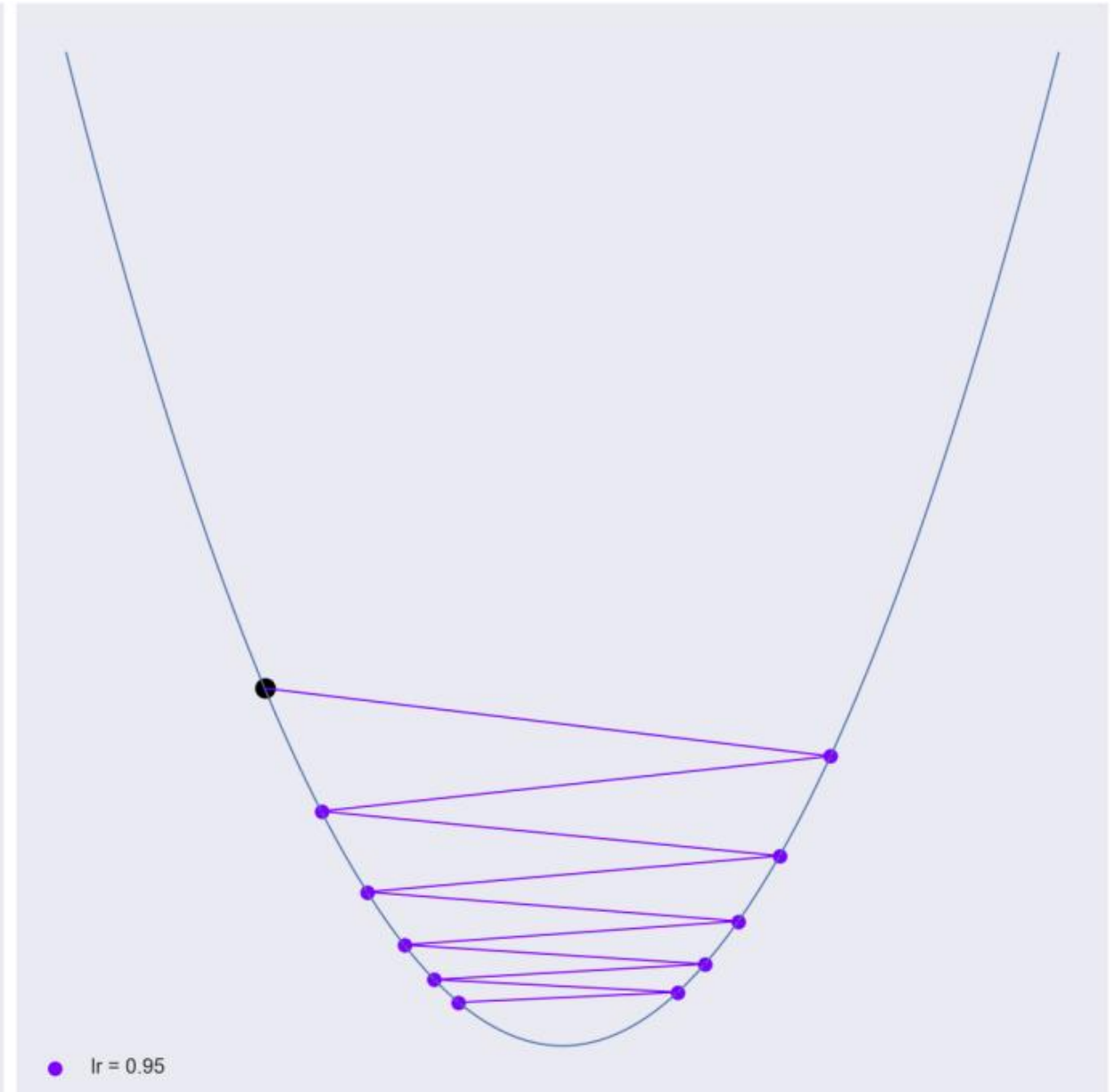
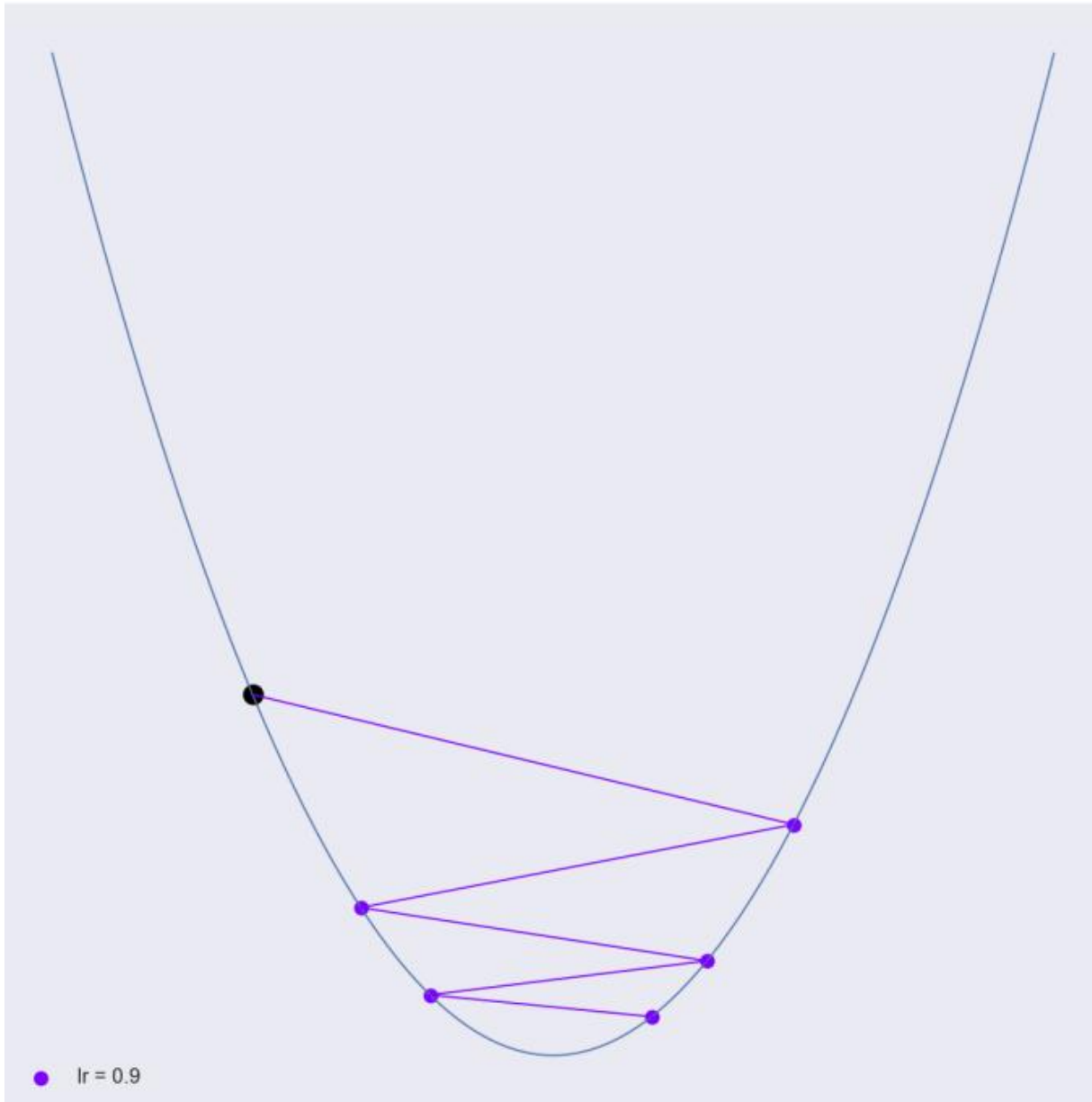
수렴

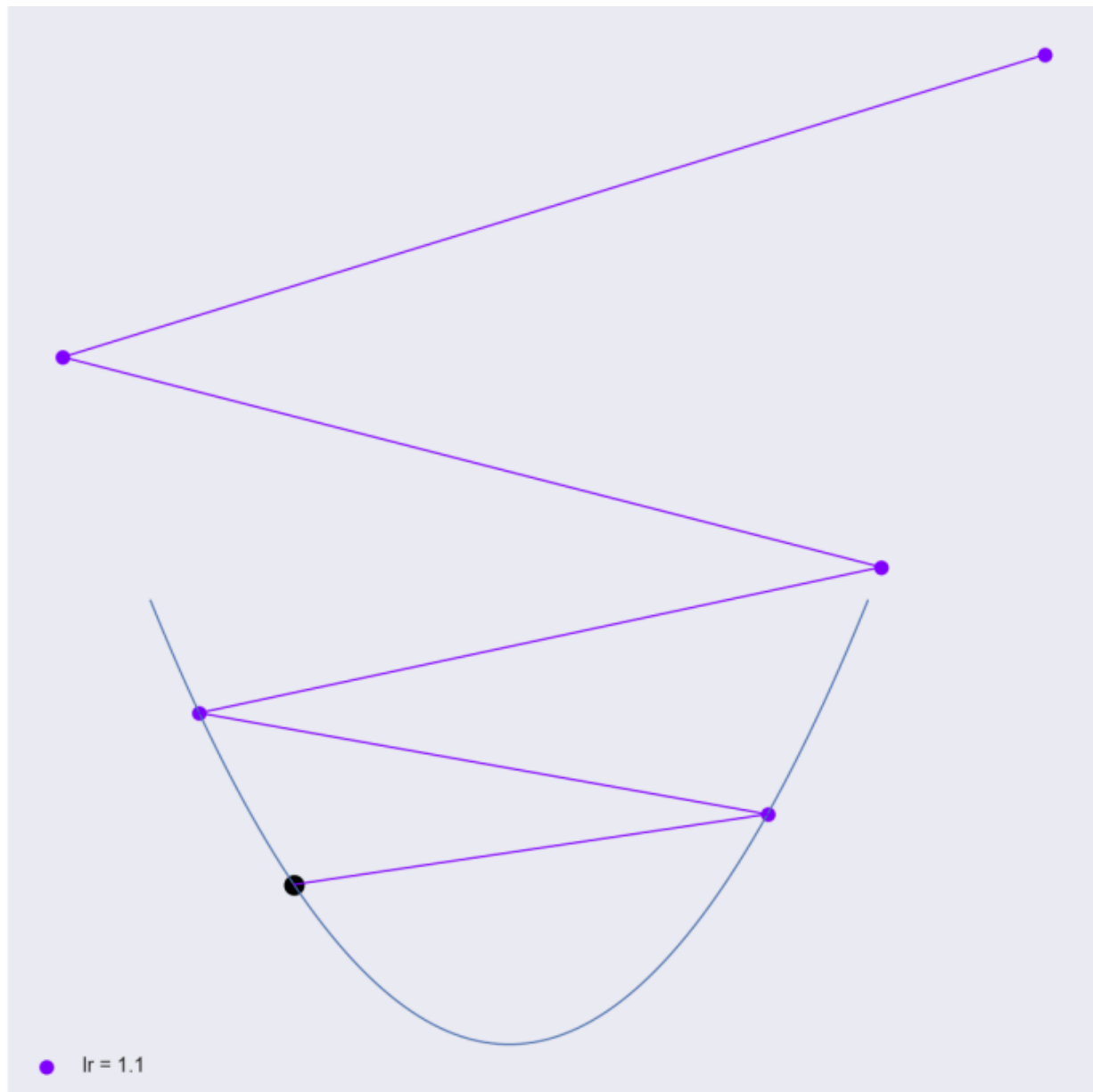
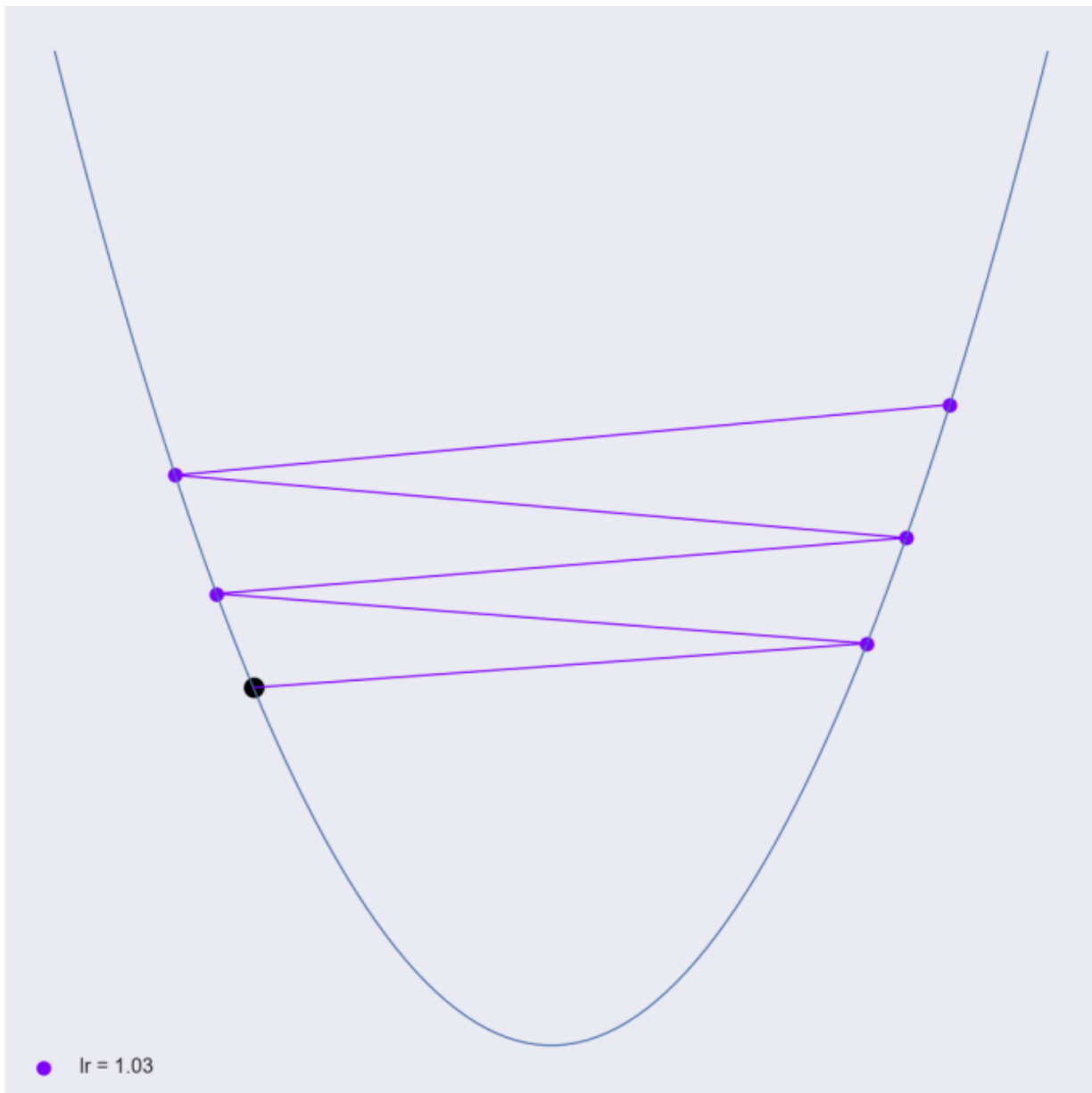


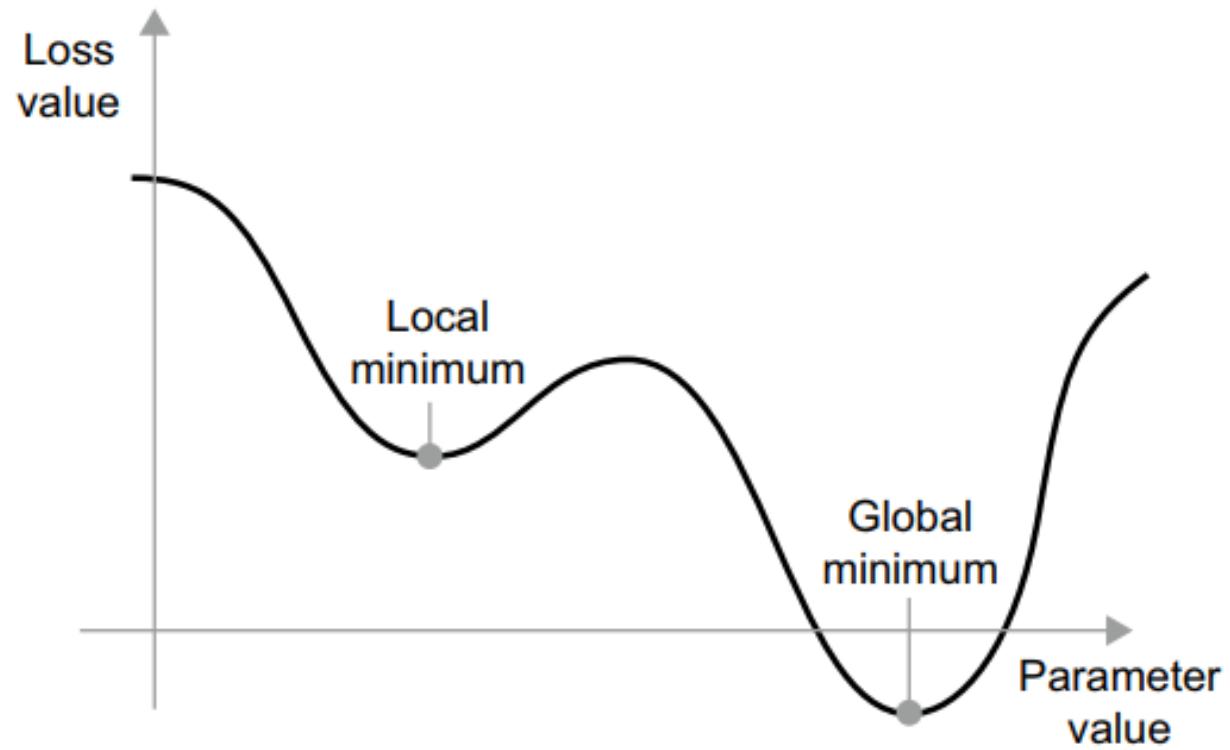
발산



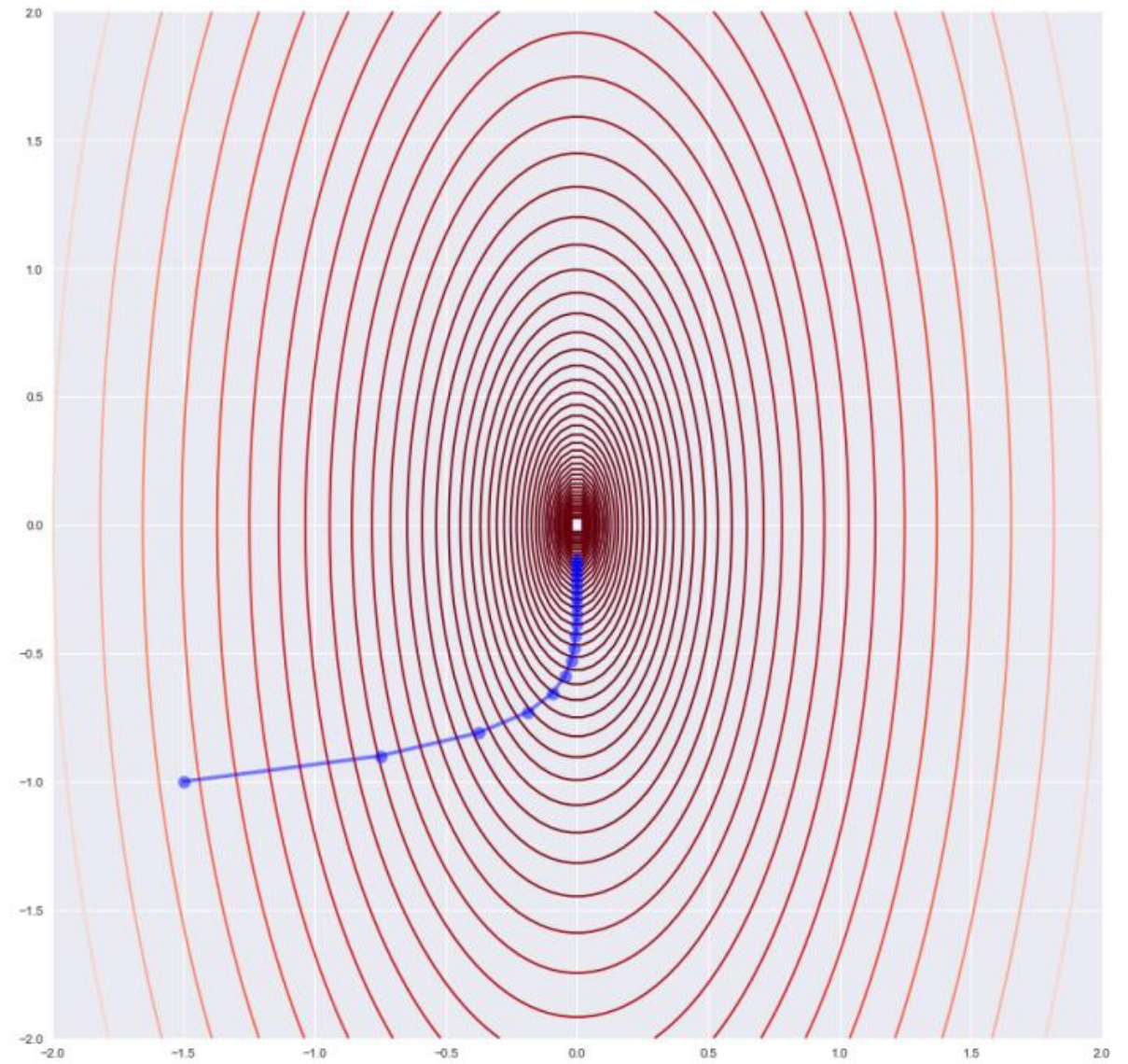
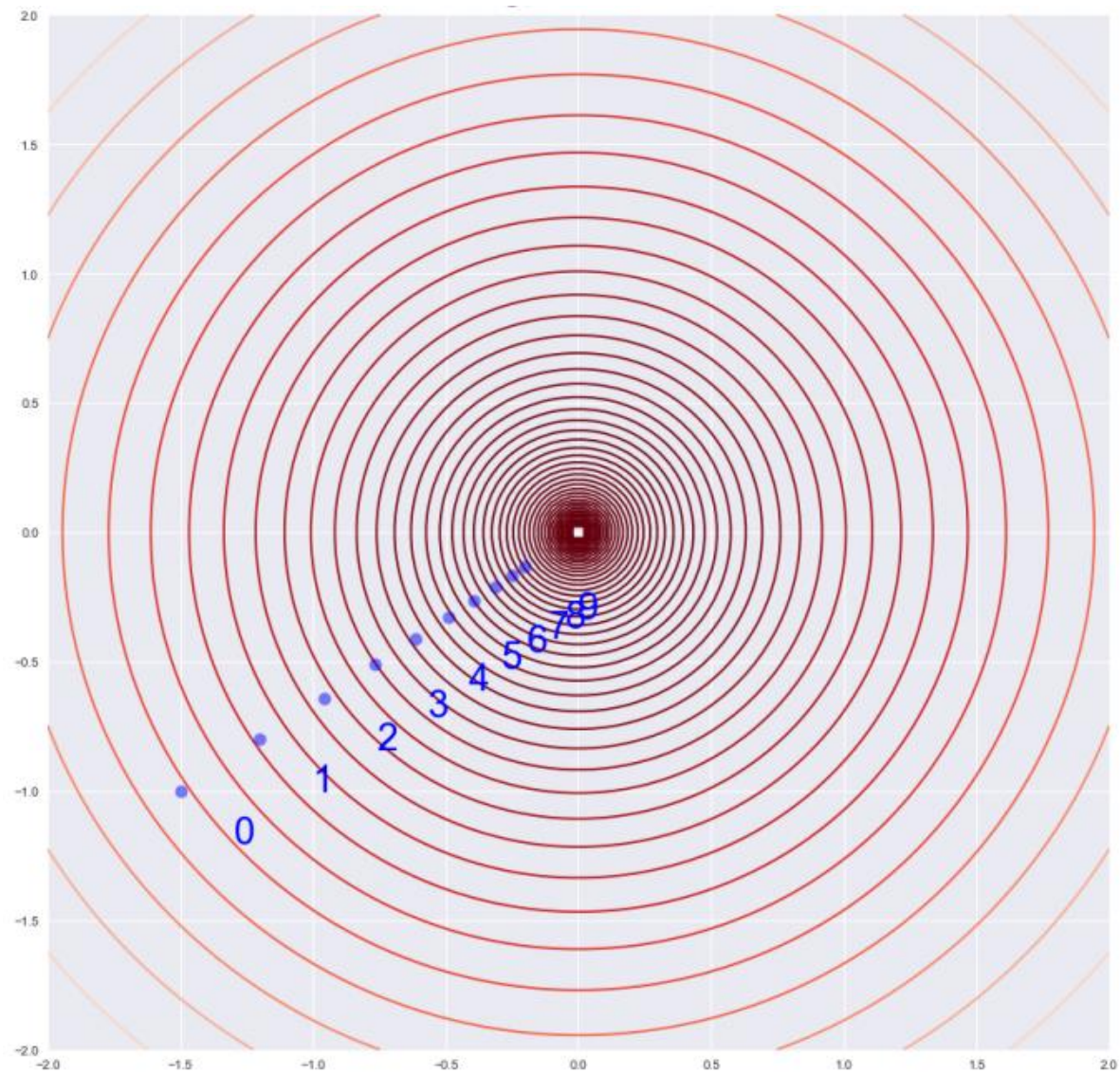


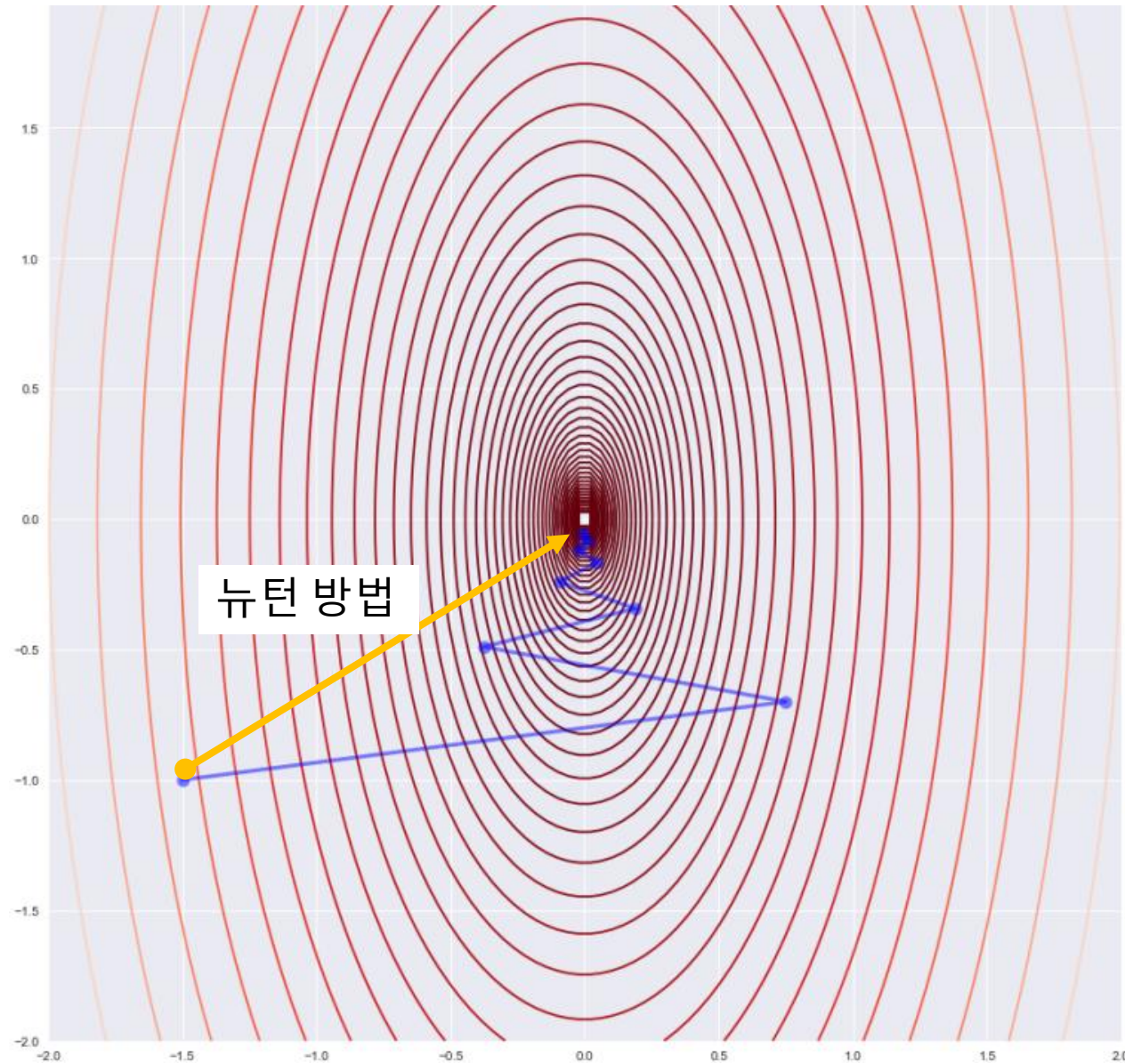






A local minimum and
a global minimum





$$\alpha = 0.15$$

코딩으로 확인하는 선형 회귀와 경사 하강법

- 012_01_Gradient Descent Method.ipynb
- 012_02_Gradient Descent Method.ipynb
- 012_03_Loss Functions.ipynb
- 012_04_Learning Rate.ipynb

참고자료

- 모두의 딥러닝, 조태호 지음, 길벗
- 케라스 창시자에게 배우는 딥러닝, 프랑소와 솔레, 길벗
- 모두를 위한 머신러닝/딥러닝 (홍콩과기대 김성훈교수)
 - <http://hunkim.github.io/ml/>
- Stanford Machine Learning
 - <http://www.holehouse.org/mlclass/>
- Machine Learning (한성대 지준교수)
 - <http://jun.hansung.ac.kr/ml>
- 수학적으로 접근하는 딥러닝 올인원 패키지, 신경식, 패스트 캠퍼스 신경식

다음 강의 주제

- 다중 선형 회귀 (multiple linear regression)